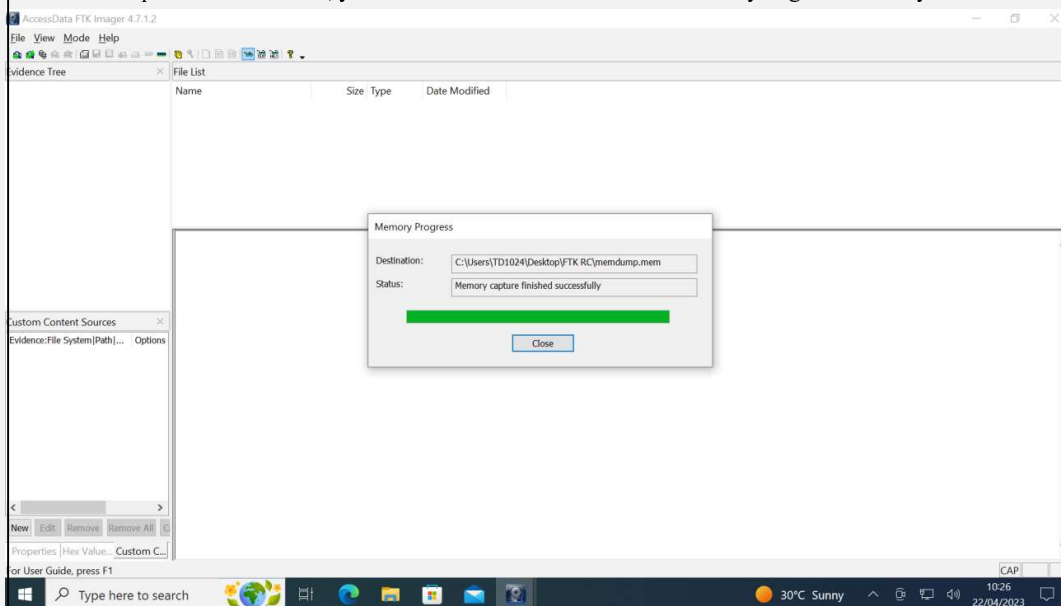


6. Once the process is finished, you can close the window and start analysing the memory file.



PRACTICAL 6

Analyzing the RAM dump using Volatility Framework to extract evidence

Volatility

Volatility is a tool for analyzing the runtime state of a system using the data found in volatile storage (RAM). It also provided a cross-platform, modular, and extensible platform to encourage further work into this exciting area of research.

Volatility is a command-based tool which is used to analyze the memory dump.

Volatility tool can be downloaded from their official website.

<https://www.volatilityfoundation.org/>

You can download the standalone version for Windows, Mac OS X and Linux. For standalone version there is no need to download any dependency files.

Requirements

- Python 2.6 or later, but not 3.0. <http://www.python.org>

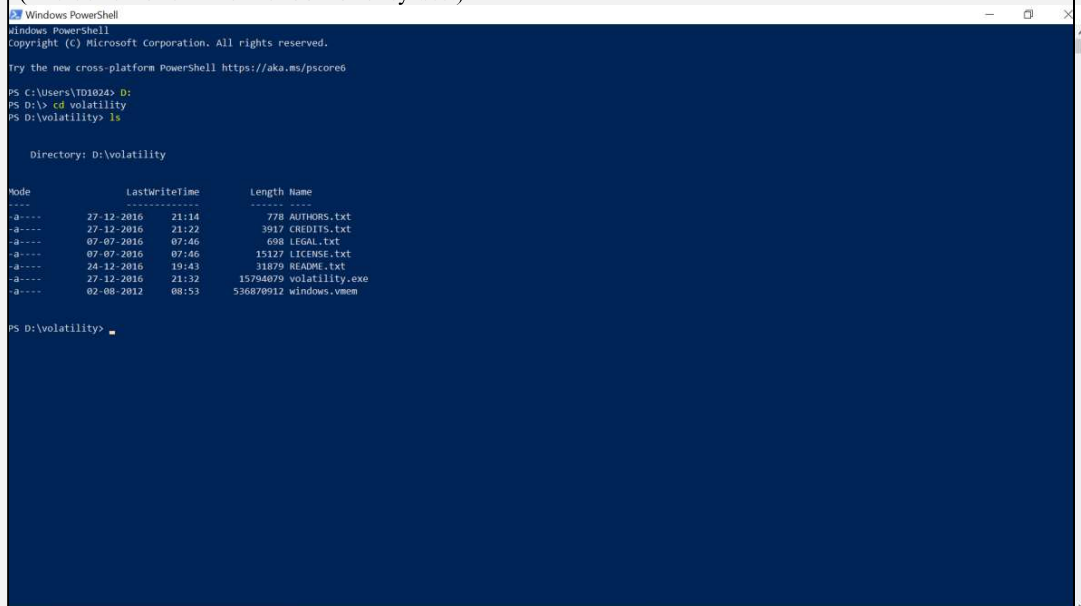
Some plugins may have other requirements which can be found at:
<https://github.com/volatilityfoundation/volatility/wiki/Installation>

In Windows System

Procedure

1. Open **Windows Powershell**
2. Go to the directory where the Volatility framework is downloaded

(It is better to rename the tool for easy use.)

A screenshot of a Windows PowerShell window. The title bar says "Windows PowerShell". The text inside shows the command prompt at "PS C:\Users\TD1024> D:", followed by "cd volatility", and then "ls". The output shows a directory listing for "Directory: D:\volatility" with columns for Mode, LastWriteTime, Length, and Name. The files listed are AUTHORS.txt, CREDITS.txt, LEGAL.txt, LICENSE.txt, README.txt, volatility.exe, and windows.vmem.

```
Windows PowerShell
Copyright (c) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\TD1024> D:
PS D:\> cd volatility
PS D:\volatility> ls

Directory: D:\volatility

Mode                LastWriteTime         Length Name
----                -
-a----          27-12-2016   21:14             776 AUTHORS.txt
-a----          27-12-2016   21:22            3917 CREDITS.txt
-a----           07-07-2016    07:46             698 LEGAL.txt
-a----           07-07-2016    07:46            15127 LICENSE.txt
-a----          24-12-2016   19:43             31879 README.txt
-a----          27-12-2016   21:32           15794079 volatility.exe
-a----           02-08-2012    08:53           536878912 windows.vmem

PS D:\volatility> .
```

3. Inside powershell, type `./volatility -h` for help or `./volatility --info` for information

```
Windows PowerShell
PS D:\volatility> ./volatility -h
Volatility Foundation Volatility Framework 2.6
Usage: Volatility - A memory forensics analysis platform.

Options:
-h, --help            list all available options and their default values.
                      Default values may be set in the configuration file
                      (/etc/volatilityrc)
--conf-file=volatilityrc
                      User based configuration file
-d, --debug           Debug volatility
--plugins=PLUGINS     Additional plugin directories to use (semi-colon
                      separated)
--info               Print information about all registered objects
--cache-directory=c:\Users\YD1024\.cache\volatility
                      Directory where cache files are stored
--cache              Use caching
--tz=TZ               Sets the (Olson) timezone for displaying timestamps
                      using pytz (if installed) or tzset
-f FILENAME, --filename=FILENAME
                      filename to use when opening an image
--profile=WinXPSP2x86
                      Name of the profile to load (use --info to see a list
                      of supported profiles)
-l LOCATION, --location=LOCATION
                      A URN location from which to load an address space
-w, --write           Enable write support
--dtb=DTB             DTB address
--shift=SHIFT         Mac KASLR shift address
--output=text         Output in this format (support is module specific, see
                      the Module Output Options below)
--output-file=OUTPUT_FILE
                      Write output in this file
-v, --verbose         Verbose information
-g KDBG, --kdbg=KDBG  Specify a KDBG virtual address (Note: for 64-bit
                      Windows a and above this is the address of
                      KdCopyDataBlock)
--force              Force utilization of suspect profile
--cookie=COOKIE       Specify the address of nt!ObHeaderCookie (valid for
                      Windows 10 only)
-k KPCR, --kpcr=KPCR  Specify a specific KPCR address

Supported Plugin Commands:
    amcache           Print AmCache information
    apihooks          Detect API hooks in process and kernel memory
```

4. To analyze a memory file, Obtain the memory file and paste it in the volatility folder (The folder you have extracted after downloading the Tool).

5. Once you done that, open PowerShell and type in the command `./volatility -f windows.vmem imageinfo`

`-f` : Filename to use when opening an image
`Imageinfo` : Identify information for the image

`Windows.vmem` is the memory dump we are using

```
Windows PowerShell
PS D:\volatility> ./volatility -f windows.vmem imageinfo
Volatility Foundation Volatility Framework 2.6
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : WinXPSP2x86, WinXPSP3x86 (Instantiated with WinXPSP2x86)
                           AS Layer1 : IA32PagedMemoryPae (Kernel AS)
                           AS Layer2 : FileAddressSpace (D:\volatility\windows.vmem)
                           PAE type : PAE
                           DTB : 0x2fe000L
                           KDBG : 0x80545ae0L
      Number of Processors : 1
      Image Type (Service Pack) : 3
                           KPCR for CPU 0 : 0xffdf000L
                           KUSER_SHARED_DATA : 0xffdf000L
      Image date and time : 2012-07-22 02:45:08 UTC+0000
      Image local date and time : 2012-07-21 22:45:08 -0400
PS D:\volatility>
```

Here we will get the information from the memory dump. The profile, Number of processes ...etc

6. We now have the computer OS from which this memory dump comes from (WinXPSP2x86). The investigation can now begin, we can specify to volatility the OS profile (`--profile=WinXPSP2x86`) and try to find what happened on the victim's computer.

7. To see what were the running processes use the *pslist* plugin.

`./volatility -f windows.vmem --profile=WinXPSP2x86 pslist`

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 pslist
Volatility Foundation Volatility Framework 2.6
Offset(V)  Name                PID  PPID  Thds  Hnds  Sess  Wow64  Start                Exit
-----
0x823c89c8 System                4    0     53   240  -----  0      0 2012-07-22 02:42:31 UTC+0000
0x822f1020 smss.exe          368   4     3    19  -----  0      0 2012-07-22 02:42:32 UTC+0000
0x822a0598 csrss.exe           584  368     9   326    0      0 2012-07-22 02:42:32 UTC+0000
0x82298700 winlogon.exe      608  368    23   519    0      0 2012-07-22 02:42:32 UTC+0000
0x81e2ab28 services.exe    652  608    16   243    0      0 2012-07-22 02:42:32 UTC+0000
0x81e2a3b8 lsass.exe      664  608    24   330    0      0 2012-07-22 02:42:32 UTC+0000
0x82311360 svchost.exe     824  652    20   194    0      0 2012-07-22 02:42:33 UTC+0000
0x81e29ab8 svchost.exe     908  652     9   226    0      0 2012-07-22 02:42:33 UTC+0000
0x823001d0 svchost.exe    1004  652    64  1118    0      0 2012-07-22 02:42:33 UTC+0000
0x821dfda0 svchost.exe    1056  652     5    60    0      0 2012-07-22 02:42:33 UTC+0000
0x82295650 svchost.exe    1220  652    15   197    0      0 2012-07-22 02:42:35 UTC+0000
0x821dea70 explorer.exe    1484 1464    17   415    0      0 2012-07-22 02:42:36 UTC+0000
0x81eb17b8 spoolsv.exe     1512  652    14   113    0      0 2012-07-22 02:42:36 UTC+0000
0x81e7bda0 reader_sl.exe  1640 1484     5    39    0      0 2012-07-22 02:42:36 UTC+0000
0x820e8da0 alg.exe        788   652     7   104    0      0 2012-07-22 02:43:01 UTC+0000
0x821fcd0 wuauclt.exe    1136 1004     8   173    0      0 2012-07-22 02:43:46 UTC+0000
0x8205bda0 wuauclt.exe    1588 1004     5   132    0      0 2012-07-22 02:44:01 UTC+0000
PS D:\volatility>
```

8. An alternative to the *pslist* plugin can be used to display the processes and their parent processes: *pstree*

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 pstree
Volatility Foundation Volatility Framework 2.6
Name                Pid  PPid  Thds  Hnds  Time
-----
0x823c89c8:System          4    0     53   240  1970-01-01 00:00:00 UTC+0000
. 0x822f1020:smss.exe      368   4     3    19  2012-07-22 02:42:31 UTC+0000
.. 0x82298700:winlogon.exe 608  368    23   519  2012-07-22 02:42:32 UTC+0000
... 0x81e2ab28:services.exe 652  608    16   243  2012-07-22 02:42:32 UTC+0000
.... 0x821dfda0:svchost.exe 1056  652     5    60  2012-07-22 02:42:33 UTC+0000
.... 0x81eb17b8:spoolsv.exe 1512  652    14   113  2012-07-22 02:42:36 UTC+0000
.... 0x81e29ab8:svchost.exe  908  652     9   226  2012-07-22 02:42:33 UTC+0000
.... 0x823001d0:svchost.exe 1004  652    64  1118  2012-07-22 02:42:33 UTC+0000
..... 0x8205bda0:wuauclt.exe 1588 1004     5   132  2012-07-22 02:44:01 UTC+0000
..... 0x821fcd0:wuauclt.exe 1136 1004     8   173  2012-07-22 02:43:46 UTC+0000
.... 0x82311360:svchost.exe  824  652    20   194  2012-07-22 02:42:33 UTC+0000
.... 0x820e8da0:alg.exe      788   652     7   104  2012-07-22 02:43:01 UTC+0000
.... 0x82295650:svchost.exe 1220  652    15   197  2012-07-22 02:42:35 UTC+0000
... 0x81e2a3b8:lsass.exe     664  608    24   330  2012-07-22 02:42:32 UTC+0000
.. 0x822a0598:csrss.exe     584  368     9   326  2012-07-22 02:42:32 UTC+0000
. 0x821dea70:explorer.exe   1484 1464    17   415  2012-07-22 02:42:36 UTC+0000
. 0x81e7bda0:reader_sl.exe  1640 1484     5    39  2012-07-22 02:42:36 UTC+0000
PS D:\volatility>
```

9. *psxview* will list processes that are trying to hide themselves while running on the computer, this plugin can be really useful.

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 psxview
Volatility Foundation Volatility Framework 2.6
Offset(P)  Name                PID  pslist  psscan  thrdproc  pspcid  csrss  session  deskthrd  ExitTime
-----
0x02498700 winlogon.exe          608  True    True     True     True   True    True     True
0x02511360 svchost.exe           824  True    True     True     True   True    True     True
0x022e8da0 alg.exe           788  True    True     True     True   True    True     True
0x020b17b8 spoolsv.exe        1512  True    True     True     True   True    True     True
0x0202ab28 services.exe    652  True    True     True     True   True    True     True
0x02495650 svchost.exe    1220  True    True     True     True   True    True     True
0x0207bda0 reader_sl.exe  1640  True    True     True     True   True    True     True
0x025001d0 svchost.exe    1004  True    True     True     True   True    True     True
0x02029ab8 svchost.exe     908  True    True     True     True   True    True     True
0x023fcd0 wuauclt.exe    1136  True    True     True     True   True    True     True
0x0225bda0 wuauclt.exe    1588  True    True     True     True   True    True     True
```

There are no processes seem to be hidden, if so you'll see "False" in the first two columns (pslist and psscan).

10. After seeing the running processes, we can also check the running sockets and open connections on the computer. To do this we'll use these different plugins: *connscan*, *netscan* and *sockets*

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 connscan
Volatility Foundation Volatility Framework 2.6
Offset(P)  Local Address          Remote Address          Pid
-----
0x02087620 172.16.112.128:1038      41.168.5.140:8080      1484
0x023a8008 172.16.112.128:1037      125.19.103.198:8080    1484
PS D:\volatility>
```

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 sockets
Volatility Foundation Volatility Framework 2.6
Offset(V)  PID  Port  Proto Protocol  Address          Create Time
-----
0x81ddb780 664  500   17  UDP         0.0.0.0          2012-07-22 02:42:53 UTC+0000
0x82240d08 1484 1038   6  TCP         0.0.0.0          2012-07-22 02:44:45 UTC+0000
0x81dd7618 1220 1900   17  UDP         172.16.112.128   2012-07-22 02:43:01 UTC+0000
0x82125610 788  1028   6  TCP         127.0.0.1         2012-07-22 02:43:01 UTC+0000
0x8219cc08 4  445    6  TCP         0.0.0.0          2012-07-22 02:42:31 UTC+0000
0x81ec23b0 908  135    6  TCP         0.0.0.0          2012-07-22 02:42:33 UTC+0000
0x82276878 4  139    6  TCP         172.16.112.128   2012-07-22 02:42:38 UTC+0000
0x82277460 4  137    17  UDP         172.16.112.128   2012-07-22 02:42:38 UTC+0000
0x81e76620 1004 123   17  UDP         127.0.0.1         2012-07-22 02:43:01 UTC+0000
0x82172808 664  0     255  Reserved    0.0.0.0          2012-07-22 02:42:53 UTC+0000
0x81e3f460 4  138    17  UDP         172.16.112.128   2012-07-22 02:42:38 UTC+0000
0x821f0630 1004 123   17  UDP         172.16.112.128   2012-07-22 02:43:01 UTC+0000
0x822cd2b0 1220 1900   17  UDP         127.0.0.1         2012-07-22 02:43:01 UTC+0000
0x82172c50 664  4500   17  UDP         0.0.0.0          2012-07-22 02:42:53 UTC+0000
0x821f0d00 4  445    17  UDP         0.0.0.0          2012-07-22 02:42:31 UTC+0000
PS D:\volatility>
```

connscan is a scanner for TCP connections, while *sockets* will print a list of open sockets and finally *netscan* (which cannot be used in our example due to the profile used) will scan a Vista (or later) image for connections and sockets.

11. To look at the last commands ran, use *cmdscan*, *consoles* and *cmdline* plugins.

Windows PowerShell

```
PS D:\volatility> ./volatility -f windows.vmem --profile=WinXPSP2x86 cmdline
Volatility Foundation Volatility Framework 2.6
*****
System pid: 4
*****
smss.exe pid: 368
Command line : \SystemRoot\System32\smss.exe
*****
csrss.exe pid: 584
Command line : C:\WINDOWS\System32\csrss.exe ObjectDirectory=\Windows SharedSection=1024,3072,512 Windows-On SubSystemType=Windows ServerDll=basesrv,1 ServerDll-winsrv:UserServerDllInitiali
zation,3 ServerDll-winsrv:ConServerDllInitialization,2 ProfileControl=Off MaxRequestThreads=16
*****
winlogon.exe pid: 600
Command line : winlogon.exe
*****
services.exe pid: 652
Command line : C:\WINDOWS\System32\services.exe
*****
lsass.exe pid: 664
```


12. There are so many other plugins which can be used for analysis of memory dump. The list of commands or plugins can be seen by using the following commands

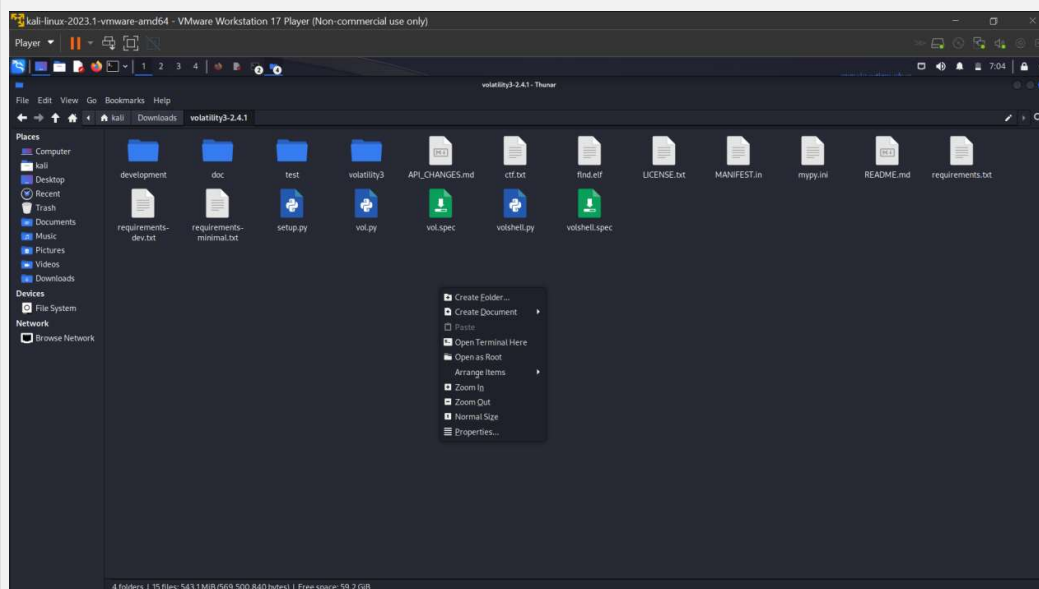
```
./volatility -h  
./volatility --info
```

In Linux Systems

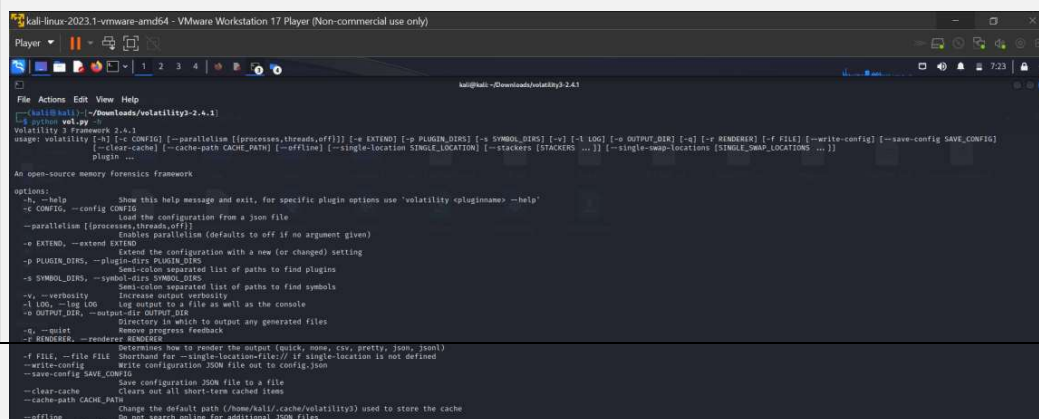
Download Volatility 3 v2.4.1 from the official website - <https://www.volatilityfoundation.org/>

Installing Volatility -

1) Right click on the extracted file and click on “open the terminal here”



5) After the required files are installed, volatility can be executed using the following command - `python vol.py -h` (-h for help).



Rest of the process is similar to that of volatility windows. If you are analysing windows RAM dump, just add *windows."plugin"* and if you are analysing linux RAM dump, just add *linux."plugin"*

Result

RAM analysis using Volatility has successfully completed.

PRACTICAL 7

Setup SIEM tool and upload the extracted logs from windows system.

Introduction

Splunk is a popular platform for analyzing machine-generated data such as logs, events, and metrics. Splunk provides powerful search and analysis capabilities that help organizations to detect and investigate security incidents, troubleshoot operational issues, and gain insights into their IT infrastructure.

In this lab, you will learn how to set up Splunk as a Security Information and Event Management (SIEM) tool. You will learn how to install and configure Splunk, create a data source, and configure security settings to collect and analyze security events. You will also learn how to create custom alerts and dashboards to monitor and visualize security events.

Prerequisites

Before you begin this lab, you should have the following:

- A virtual or physical machine running a supported operating system (Windows, Linux, or macOS).
- Sufficient resources (CPU, RAM, and disk space) to run Splunk.
- Administrative privileges on the machine.
- Internet connectivity to download Splunk software.

Part 1: Installing and Configuring Splunk

- Download the Splunk software from the official website (https://www.splunk.com/en_us/download.html).
- Follow the instructions to install Splunk on your machine. You may need to provide administrative credentials and choose the appropriate installation options.
- After installation, launch the Splunk web interface by opening a web browser and navigating to <http://localhost:8000>. You should see the Splunk login page.
- Login to Splunk with the default credentials (username: admin, password: changeme).
- After login, you will be prompted to change the password. Choose a strong password and remember it for future logins.
- Once you have changed the password, you will see the Splunk Home screen. From here, you can start configuring Splunk to collect and analyze security events.

Part 2: Creating a Data Source

- To create a data source, go to the Splunk Home screen and click on "Add Data" in the "Getting Started" section.
- Select the data source that you want to collect, such as log files or network traffic.
- Follow the instructions to configure the data source. You may need to provide authentication credentials, specify the log format, and set up filters to exclude irrelevant data.
- Once you have configured the data source, Splunk will start indexing the data and making it available for search and analysis.

Part 3: Configuring Security Settings

- To configure security settings, go to the Splunk Home screen and click on "Settings" in the top menu.
- Click on "Security" to access the security settings.
- Configure the security settings according to your organization's requirements. This may include configuring authentication and authorization, enabling SSL/TLS encryption, and setting up audit logging.
- Test the security settings by logging out of Splunk and logging back in with a different user account.

Part 4: Creating Custom Alerts and Dashboards

- To create custom alerts and dashboards, go to the Splunk Home screen and click on "Create" in the top menu.
- Select "Alert" to create a new alert or "Dashboard" to create a new dashboard.
- Follow the instructions to configure the alert or dashboard. You can specify the search criteria, set up notifications, and customize the layout and visualization options.
- Once you have created the alert or dashboard, you can view it on the Splunk Home screen and monitor it for security even

Conclusion

In this lab, you learned how to set up Splunk as a Security Information and Event Management

PRACTICAL 8

Installing Wireshark and creating PCAP files for analysis. Application of Wireshark search filters.

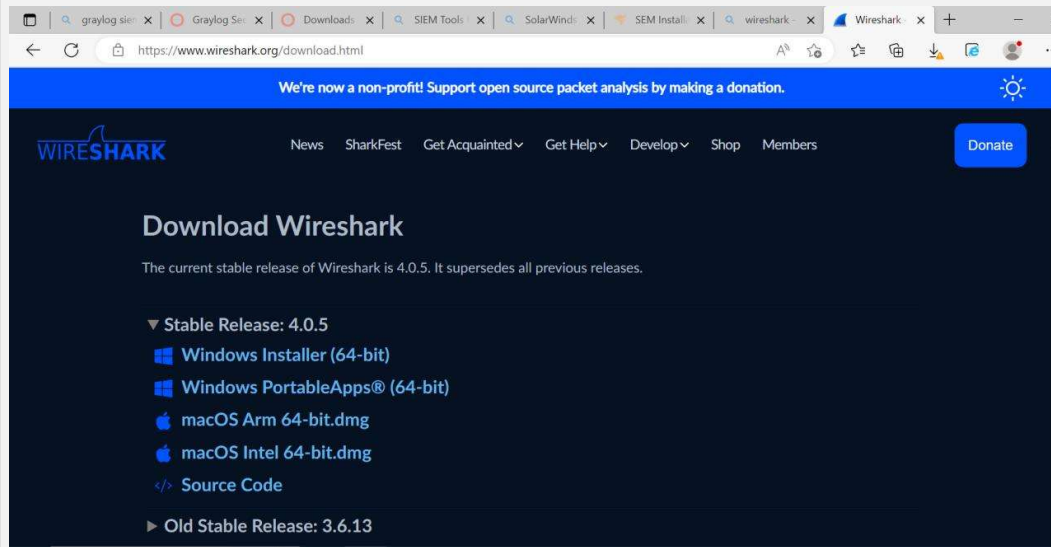
Wireshark

Wireshark is the world's foremost network protocol analyzer. It lets you see what's happening on your network at a microscopic level.

Installing Wireshark

Wireshark should be downloaded from the official website given below

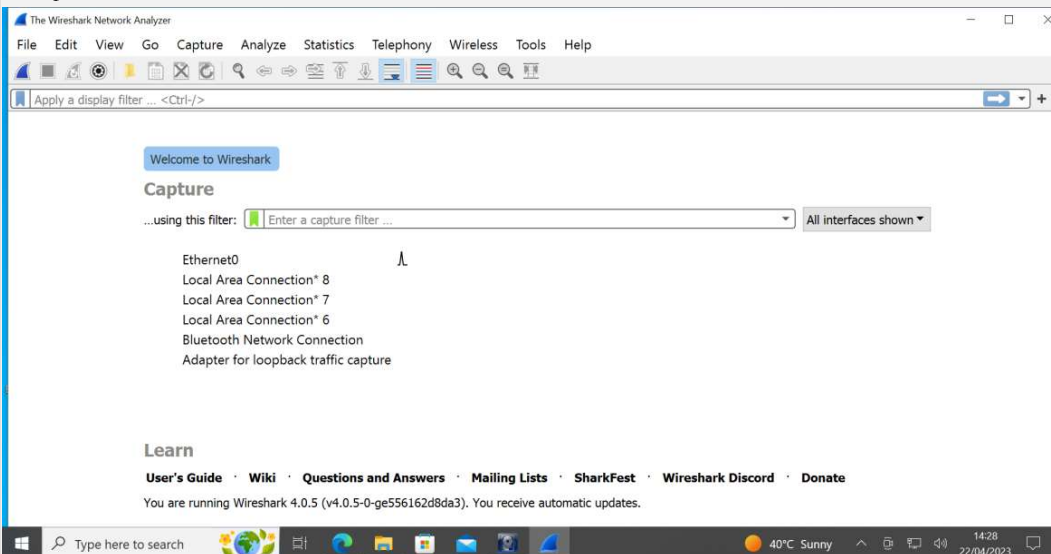
<https://www.wireshark.org/download.html>



Download the stable version of Wireshark from the given list.

Creating PCAP files for Analysis

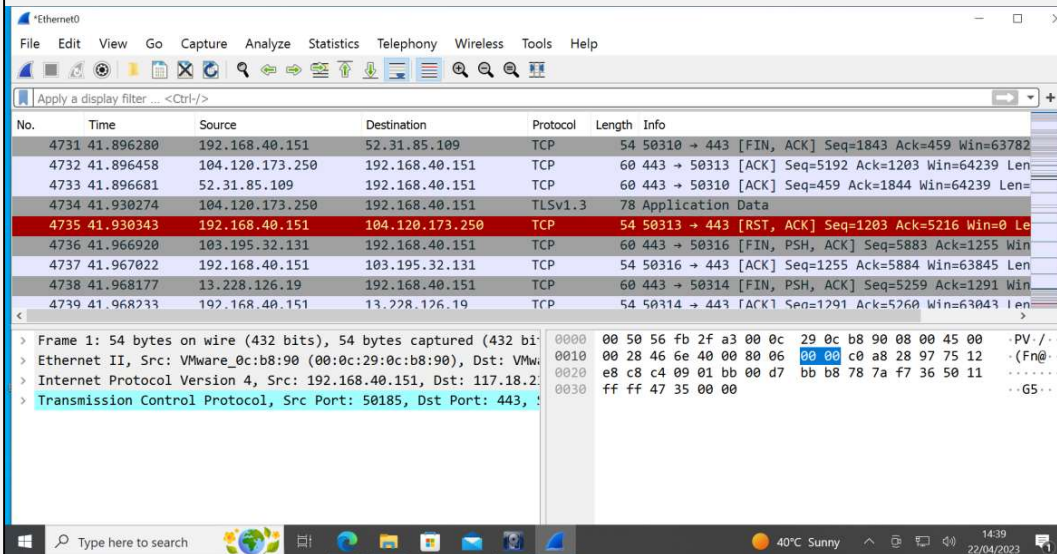
1. Open Wireshark.



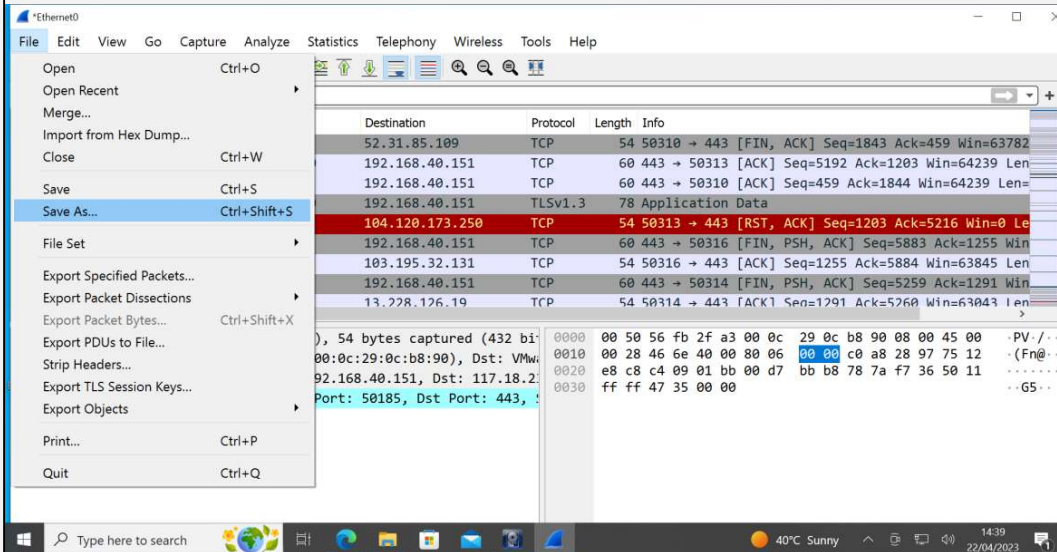
This is how the Wireshark interface looks like.

2. Select the Network you are connected to
3. Wireshark will now start to capture the packets.
4. Open Browser and search for any websites, keeping wireshark in the background.

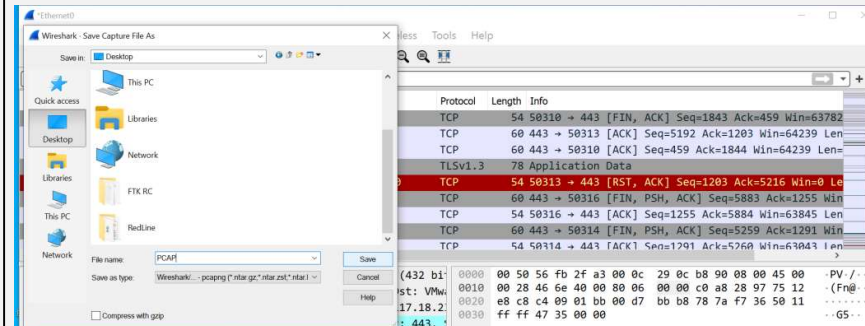
5. Wireshark will now capture the packets that is being send from the client to server and vice versa.



6. To create PCAP file, go to File option

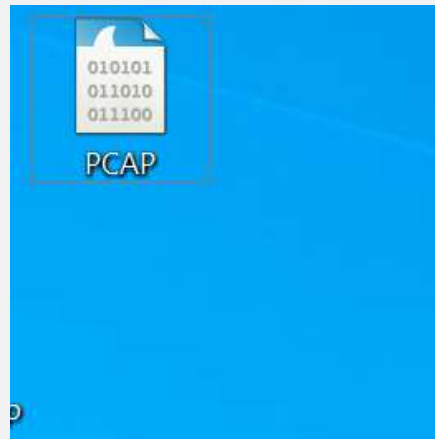


7. Click on Save As....



8. Type the File name and click on Save. The PCAP file will be saved on the destined Folder.

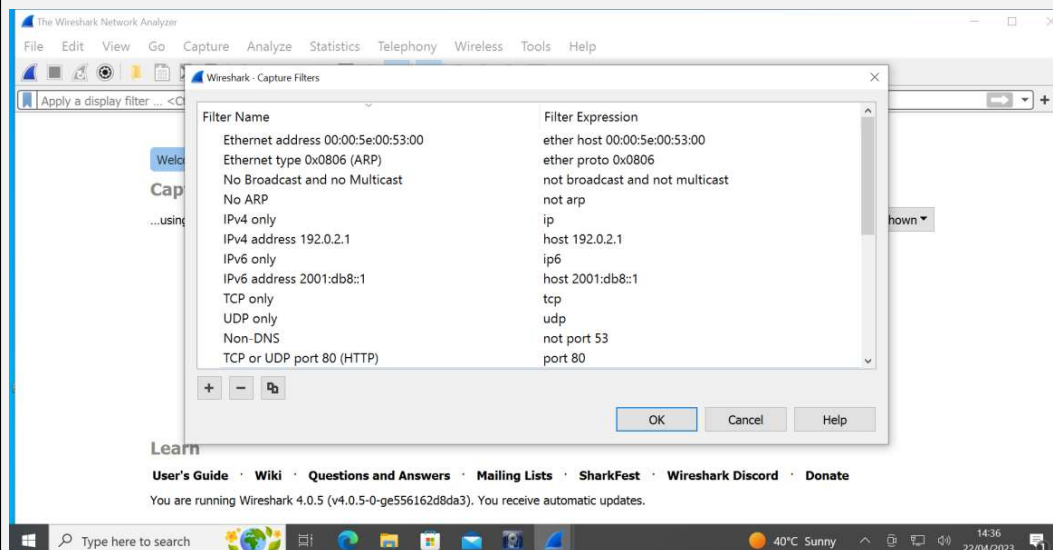




Application of Capture Filters

Capture filters limit the captured packets by the chosen filter. If the packets don't match the filter, Wireshark won't save them. Examples of capture filters include:

- host IP-address: This filter limits the captured traffic to and from the IP address
- net 192.168.0.0/24: This filter captures all traffic on the subnet
- dst host IP-address: Capture packets sent to the specified host
- port 53: Capture traffic on port 53 only
- port not 53 and not Arp: Capture all traffic except DNS and ARP traffic

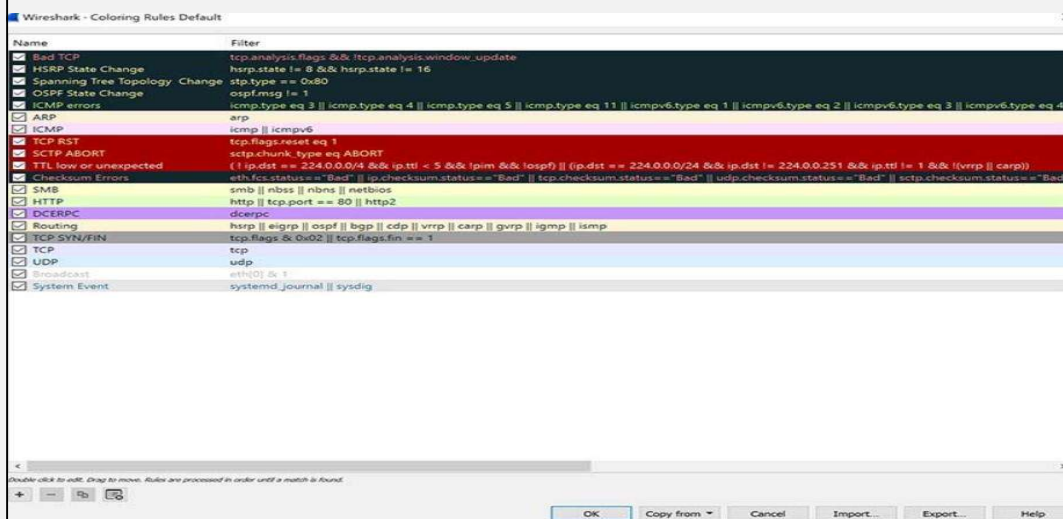


Color Coding in Wireshark

The wireshark tries to help you identify packet types by applying common-sense colour coding.

Color in Wireshark	Packet Type
Light purple	TCP
Light blue	UDP
Black	Packets with errors
Light green	HTTP traffic

You can view this by going to View >> Coloring Rules.



RESULT:-Packets were captured and analysis was studied.

PRACTICAL 9

Network malware logs analysis CTF (using Wireshark)

Introduction

Wireshark can not only be used for network analysis but also it can be used for malware analysis. It helps the investigator to determining whether the user has downloaded any malicious file from the internet or not by analyzing the network logs.